

Si por **FloorplanVLM** te refieres al paper de Beike publicado en 2026, mi recomendación es **no intentar reemplazar ArchiCheck por FloorplanVLM**, sino usarlo como una tercera fuente de evidencia estructural dentro de tu pipeline actual. El modelo reformula la vectorización como un problema "imagen → JSON estructurado" (muros, puertas, ventanas, recintos), en lugar de segmentación o detección aislada de objetos. [\[arxiv.org\]](#), [\[arxiv.org\]](#)

Lo primero: una advertencia importante

El paper reporta resultados muy fuertes (92.52% IoU en muros exteriores), pero fueron obtenidos con:

- Dataset propietario **Floorplan-2M**
- Subconjunto **HQ-300K**
- Infraestructura muy superior (32× H200)
- Datos que no son públicos actualmente. [\[arxiv.org\]](#), [\[arxiv.org\]](#)

Por lo tanto:

No asumiría que descargar un repositorio llamado "FloorPlanVLM" reproducirá esos resultados.

El propio repositorio comunitario reconoce explícitamente que no puede reproducir los números del paper porque los datos de entrenamiento no son públicos. [\[github.com\]](#)

Cómo lo integraría en ArchiCheck

No aquí:

```
1 PDF
2 ↓
3 FloorplanVLM
4 ↓
5 Reglas OGUC
```

Sí aquí:

```
1 PDF Vectorial
2 └─ Extractor geométrico propio
3 └─ Claude / GPT-4o
4 └─ FloorplanVLM
5
6           ↓
7
8 Fusionador de evidencias
```

```
9
10         ↓
11
12   Geometría validada por arquitecto
13
14         ↓
15
16   Motor normativo
```

Uso concreto 1 (el que más me gusta)

FloorplanVLM como detector de discrepancias

Hoy tienes:

```
1  Geometría PDF
```

y después

```
1  Validación humana
```

Yo agregaría:

```
1  Raster 300 dpi
2      ↓
3  FloorplanVLM
4      ↓
5  JSON estructural
```

Ejemplo:

Extractor PDF:

```
1  {
2    "doors": 12
3  }
```

FloorplanVLM:

```
1  {
2    "doors": 19
3  }
```

Diferencia:

```
1  +7 puertas faltantes
```

Justamente el tipo de problema que describiste.

En lugar de buscar puertas con filtros nuevos:

```
1 "Hay evidencia conflictiva."
```

y lo marcas para revisión.

Uso concreto 2

Generador de hipótesis

Hoy cuando fallan:

```
1 ¿por qué no apareció esta puerta?
```

requiere horas de debugging.

Con FloorplanVLM:

```
1 Puerta detectada.  
2 Centro aproximado:  
3 (1245, 882)
```

Entonces disparas una búsqueda localizada:

```
1 buscar_arcos(  
2     radio=1 m,  
3     centro=(1245,882)  
4 )
```

FloorplanVLM no entrega la geometría final.

Entrega:

```
1 candidato geométrico
```

para que tu extractor vectorial lo confirme.

Eso reduce muchísimo el espacio de búsqueda.

Uso concreto 3

Reconstrucción de recintos

Para mí éste es el mejor fit.

Tu pipeline actual obtiene recintos mediante:

```
1 adaptiveThreshold  
2 +
```

```
3 connectedComponents
```

Funciona.

Pero es sensible al estilo gráfico.

FloorplanVLM produce explícitamente:

```
1 {  
2   "rooms": [...]  
3 }
```

como secuencias estructuradas. [\[arxiv.org\]](#), [\[arxiv.org\]](#)

Yo lo usaría para:

Comparación topológica

Extractor:

```
1 27 recintos
```

FloorplanVLM:

```
1 31 recintos
```

Revisión automática.

Lo que NO haría

No intentaría extraer medidas normativas directamente del VLM.

No confiaría en:

```
1 width: 0.92
```

como dato oficial.

Tu extractor vectorial es más confiable para dimensiones.

Mantendría:

```
1 FloorplanVLM  
2 =  
3 estructura
```

```
1 PDF vectorial  
2 =  
3 mediciones
```

Cómo entrenarlo para Chile

Aquí está el verdadero valor.

Ya tienes algo extremadamente raro:

Ground Truth validado

La mayoría de los papers no tienen:

```
1 PDF original
2 +
3 geometría corregida por arquitecto
4 +
5 resultado normativo
```

Ustedes sí.

Ese dataset vale oro.

Yo construiría un formato:

```
1 {
2   "walls": [...],
3   "doors": [...],
4   "windows": [...],
5   "rooms": [...]
6 }
```

derivado de la corrección manual.

Después fine-tuning.

Estrategia de entrenamiento que seguiría

Etapa 1

CubiCasa

Objetivo:

```
1 aprender simbología base
```

Etapa 2

Planos chilenos reales

Objetivo:

1 adaptación de dominio

Etapa 3

Planos problemáticos

Los que hoy te rompen el pipeline.

Ejemplos:

- puertas invisibles
- hatch agresivo
- CAD antiguo
- exportadores PDF raros

Entrenar desproporcionadamente sobre esos casos.

Arquitectura que creo que tiene más potencial

Si mañana liderara ArchiCheck, probaría:

1	PDF	VECTORIAL
2		+
3	MLSTRUCT	-FP
4		+
5	FloorplanVLM	

donde:

PDF vectorial

Fuente de verdad geométrica.

MLSTRUCT-FP

Prior chileno.

FloorplanVLM

Prior estructural/topológico.

Fusionador

Produce:

1 confidence:

```
2 {  
3   walls: 0.97,  
4   doors: 0.74,  
5   windows: 0.88,  
6   rooms: 0.93  
7 }
```

y sólo muestra al arquitecto los conflictos.

Prioridad de implementación

1. **No entrenar nada aún.**
2. Correr un checkpoint comunitario de FloorplanVLM sobre 50 planos chilenos.
[\[github.com\]](https://github.com), [\[huggingface.co\]](https://huggingface.co)
3. Medir:
 - recall de puertas
 - recall de ventanas
 - error topológico de recintos
4. Usarlo inicialmente sólo como sistema de discrepancias.
5. Si supera a CubiCasa5K en recall estructural, recién ahí invertir en fine-tuning.

Para ArchiCheck, veo mucho más valor en FloorplanVLM como **segunda opinión estructural que detecta cosas que el extractor vectorial omitió**, que como reemplazo completo de tu pipeline actual. Eso aprovecha lo mejor de ambos mundos: precisión métrica de PDF vectorial y robustez semántica de un VLM.